ĐẶC TẢ THIẾT KẾ gemm_local_sram

Module của `gemm_accel_top` · instance `u_sram`

Mục lục

1 Tổng quan **2**

1.1 Tính năng chính . . . 2
1.2 Tham số . . . 2
1.3 Tín hiệu giao tiếp . . . 2
1.4 Điều kiện phục vụ đồng thời . . . 5

2 Đặc tả thanh ghi **5**

3 Mô tả chức năng **6**

3.1 Bố cục logic . . . 6
3.2 Tổ chức vật lý và ánh xạ . . . 6
3.3 Module con . . . 8
3.4 Timing . . . 10
3.5 Reset, hiện thực, kiểm chứng . . . 11

1 Tổng quan

1.1 Tính năng chính

Scratchpad giữ bốn buffer logic A, B, C, BIAS, mỗi buffer có hai bank (ping-pong, NB = 2). Phía tính (array, VPU) cần đọc hoặc ghi 8 phần tử trong một chu kỳ, nên mỗi bank được chia thành nhiều **sub-bank** một cổng sao cho mỗi truy cập song song chạm các sub-bank khác nhau. Phía DMA đọc hoặc ghi một word mỗi chu kỳ. Hai phía dùng hai bank khác nhau, nên không bao giờ tranh nhau một sub-bank.

Chức năng

- Mỗi buffer có hai bank. Mỗi bank: A 256 word, B 256 word, C 1024 word, BIAS 32 word, tức 1568 word mỗi bank; tổng 3136 word = 12,25 KiB
- Mọi cổng đọc có độ trễ cố định đúng 1 chu kỳ; mọi lệnh ghi có hiệu lực ở cạnh clock cuối chu kỳ có lệnh; không có backpressure (`o_dw_ready` = `o_dr_ready` = 1 hằng số)
- Mỗi sub-bank là SRAM một cổng đồng bộ. Bank b của buffer X chỉ bị truy cập bởi một phía trong một chu kỳ; hai bank khác nhau được truy cập độc lập cùng chu kỳ
- Mỗi chu kỳ phía tính truy cập đồng thời A, B, BIAS, C; phía DMA truy cập đồng thời một cổng ghi (A, B hoặc BIAS) và một cổng đọc (C)
- Nội dung RAM không được khởi tạo bởi `rst_n` hay `soft_clr`; nội dung sau reset không xác định

1.2 Tham số

Bảng 1.1: Tham số

Tham số	Giá trị	Ý nghĩa
`P`	8	localparam cố định, bằng `PE_ARRAY_SIZE` của top (chỉ nhận 8): số lane của vector phía tính
`NB`, `Q`, `LP`	2, 2, 3	localparam. NB = 2 luôn luôn (hai bank mỗi buffer). Q = P/4 = 2 (số sub-bank của B); LP = $\log_2 P$ = 3

QUYẾT ĐỊNH BASELINE
D-04 (P = 8 cố định). Q = P/4 = 2, LP = 3; bus vector A, B rộng 8P = 64 bit, bus vector BIAS và C rộng 32P = 256 bit. Mọi công thức trong tài liệu này được viết cho P = 8 và không có biến thể P = 4.

1.3 Tín hiệu giao tiếp

Bảng 1.2: Tín hiệu giao tiếp của `gemm_local_sram`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Cạnh nhả reset được đồng bộ với clock tại top trước khi đưa tới các module.
`i_soft_clr`	1	Input	Xung xóa đồng bộ dài một chu kỳ do Main FSM phát. Xóa trạng thái điều khiển tại cạnh lên; phạm vi xóa dữ liệu được quy định trong mục Reset.

Cổng ghi phía DMA (LOAD_A, LOAD_B, LOAD_BIAS)

Bảng 1.2: Tín hiệu giao tiếp của `gemm_local_sram` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_dw_valid`	1	Input	Yêu cầu ghi một word 32 bit vào scratchpad. Ghi được nhận khi valid và ready cùng bằng 1 tại cạnh lên.
`i_dw_buf`	2	Input	Chọn buffer đích cho word ghi. Giá trị 0: A. Giá trị 1: B. Giá trị 2: BIAS; giá trị 3 không hợp lệ.
`i_dw_bank`	1	Input	Chọn bank đích cho word ghi. Hợp lệ cùng yêu cầu ghi; nhận giá trị 0 hoặc 1.
`i_dw_addr`	10	Input	Địa chỉ word logic trong buffer đích. A/B dùng miền 0 đến 255; BIAS dùng miền 0 đến 31. Hợp lệ cùng yêu cầu ghi.
`i_dw_data`	32	Input	Word dữ liệu 32 bit cần ghi vào scratchpad. Hợp lệ cùng yêu cầu ghi; DMA giữ nguyên thứ tự byte của dữ liệu DRAM.
`o_dw_ready`	1	Output	Báo scratchpad nhận word ghi. Luôn bằng 1 (không có backpressure); giao dịch vẫn được xác định bởi valid và ready.
Cổng đọc phía DMA (STORE_C)			
`i_dr_valid`	1	Input	Yêu cầu đọc một word từ buffer C. Yêu cầu được nhận tại cạnh lên khi valid và ready cùng bằng 1.
`i_dr_bank`	1	Input	Chọn bank nguồn của buffer C. Hợp lệ cùng yêu cầu đọc; nhận giá trị 0 hoặc 1.
`i_dr_addr`	10	Input	Địa chỉ word logic cần đọc trong buffer C. Hợp lệ cùng yêu cầu đọc; miền địa chỉ phụ thuộc bố cục INT32 hoặc INT8 của C.
`o_dr_ready`	1	Output	Báo scratchpad nhận yêu cầu đọc C. Luôn bằng 1 (không có backpressure).
`o_dr_rvalid`	1	Output	Báo word đọc trả về hợp lệ. Xuất hiện đúng một chu kỳ sau yêu cầu được nhận; mỗi yêu cầu có đúng một phản hồi, theo thứ tự.
`o_dr_rdata`	32	Output	Word dữ liệu 32 bit đọc từ buffer C. Bên nhận lấy dữ liệu khi read rvalid = 1; không có ready cho kênh trả dữ liệu.
Bank phía tính (từ FSM)			
`i_comp_bank_a`	1	Input	Chọn bank của buffer A dành cho phía tính. Giữ nguyên trong suốt COMPUTE.
`i_comp_bank_b`	1	Input	Chọn bank của buffer B dành cho phía tính. Giữ nguyên trong suốt COMPUTE.
`i_comp_bank_bias`	1	Input	Chọn bank của buffer BIAS dành cho phía tính. Giữ nguyên trong suốt COMPUTE.
`i_comp_bank_c`	1	Input	Chọn bank của buffer C dành cho phía tính. Giữ nguyên trong suốt COMPUTE.
`i_comp_active`	1	Input	Báo COMPUTE đang hoạt động. SRAM dùng tín hiệu này để kiểm tra quyền truy cập bằng assertion; tín hiệu không thay thế các enable đọc/ghi.
Cổng đọc A, B (từ array)			

Bảng 1.2: Tín hiệu giao tiếp của `gemm_local_sram` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_a_rd_en`	1	Input	Cho phép đọc một vector A. Địa chỉ hợp lệ trong chu kỳ enable = 1; dữ liệu trả về đúng một chu kỳ sau.
`i_a_rd_row`	5	Input	Hàng đầu i0 của vector A. Là bội của 8; hợp lệ cùng read enable.
`i_a_rd_k`	5	Input	Chỉ số k của cột A cần đọc. Hợp lệ cùng read enable; phải nhỏ hơn K.
`o_a_rd_data`	64	Output	Vector 8 byte INT8 của A. Lane r tại bit [8r +: 8] chứa A[i0+r][k]. Hợp lệ một chu kỳ sau yêu cầu đọc; lane ngoài miền tác vụ chứa giá trị bất kỳ.
`i_b_rd_en`	1	Input	Cho phép đọc một vector B. Địa chỉ hợp lệ trong chu kỳ enable = 1; dữ liệu trả về đúng một chu kỳ sau.
`i_b_rd_k`	5	Input	Chỉ số k của hàng B cần đọc. Hợp lệ cùng read enable; phải nhỏ hơn K.
`i_b_rd_col`	5	Input	Cột đầu j0 của vector B. Là bội của 8; hợp lệ cùng read enable.
`o_b_rd_data`	64	Output	Vector 8 byte INT8 của B. Lane c tại bit [8c +: 8] chứa B[k][j0+c]. Hợp lệ một chu kỳ sau yêu cầu đọc; lane ngoài miền tác vụ chứa giá trị bất kỳ.
Cổng BIAS và C (từ VPU)			
`i_bias_rd_en`	1	Input	Yêu cầu đọc 8 phần tử bias. Bằng 1 khi đoạn hàng hợp lệ và hậu xử lý được bật; dữ liệu về sau một chu kỳ.
`i_bias_rd_col`	5	Input	Cột đầu j0 của vector bias. Là bội của 8; hợp lệ cùng bias read enable.
`o_bias_rd_data`	256	Output	Vector 8 giá trị bias INT32. Lane c tại bit [32c +: 32] chứa bias[j0+c]. Hợp lệ đúng một chu kỳ sau yêu cầu; lane bị mask chứa giá trị bất kỳ.
`i_c_wr_en`	1	Input	Cho phép ghi một đoạn hàng kết quả vào buffer C tại cạnh lên. Các trường fmt, row, col, data và mask phải hợp lệ trong chu kỳ này.
`i_c_wr_fmt`	1	Input	Chọn định dạng phần tử ghi C. Giá trị 0: INT32, mỗi phần tử là một word. Giá trị 1: INT8, mỗi phần tử là một byte. Hợp lệ cùng write enable.
`i_c_wr_row`	5	Input	Chỉ số hàng i của buffer C cần ghi. Hợp lệ cùng write enable.
`i_c_wr_col`	5	Input	Cột đầu j0 của đoạn hàng cần ghi. Là bội của 8; hợp lệ cùng write enable.
`i_c_wr_data`	256	Input	Dữ liệu ghi đoạn hàng C. Với fmt = 0, phần tử c nằm tại bit [32c +: 32]. Với fmt = 1, phần tử c nằm tại bit [8c +: 8] và phần cao còn lại bằng 0. Hợp lệ cùng write enable.
`i_c_wr_mask`	8	Input	Cho phép ghi theo phần tử. Bit c = 1 ghi phần tử c theo định dạng fmt; bit c = 0 giữ nguyên phần tử đó. Hợp lệ cùng write enable.

Giao tiếp

- `o_dw_ready` và `o_dr_ready` luôn bằng 1. Lệnh chỉ được nhận khi valid và ready cùng bằng 1. Mỗi chu kỳ nhận một word ghi và một yêu cầu đọc; mỗi yêu cầu đọc có đúng một phản hồi theo thứ tự
- `o_a_rd_data`, `o_b_rd_data`, `o_bias_rd_data` hợp lệ ở đúng chu kỳ sau chu kỳ có `*_rd_en`; ở chu kỳ khác giá trị tùy ý. `o_dr_rvalid` là thanh ghi
- Lane ứng với hàng hoặc cột ngoài vùng hợp lệ của tác vụ mang nội dung tùy ý

1.4 Điều kiện phục vụ đồng thời

Trong một chu kỳ SRAM phải phục vụ đồng thời một lần đọc A (8 byte từ 8 hàng), một lần đọc B (8 byte liền nhau), một lần đọc BIAS (8 word) và một lần ghi C (8 phần tử có mask). Hai phía tính và DMA chạy đồng thời trên các bank khác nhau.

Bảng 1.3: Truy cập song song và sub-bank bị chạm (P = 8)

Truy cập	Sub-bank chạm	Vì sao không trùng
Đọc A (`i_a_rd_*`)	8 của A[b]	i_0 là bội của 8, nên 8 hàng $i_0 + r$ có i mod 8 khác nhau
Đọc B (`i_b_rd_*`)	2 của B[b]	j_0 là bội của 8, nên hai word $jw = j_0/4,\ j_0/4 + 1$ có jw mod 2 khác nhau
Đọc BIAS (`i_bias_rd_*`)	8 của BIAS[b]	8 cột $j_0 + c$ có j mod 8 khác nhau
Ghi C INT32 (`i_c_wr_*`, fmt = 0)	8 của C[b]	8 cột $j_0 + c$ có j mod 8 khác nhau
Ghi C INT8 (fmt = 1)	2 của C[b]	hai word liền nhau jw có jw mod 8 khác nhau
DMA ghi A, B, BIAS	1	một word mỗi chu kỳ
DMA đọc C	1	một word mỗi chu kỳ

- Phía tính chạm bank `i_comp_bank_*` của cả bốn buffer; phía DMA chạm buffer và bank do `i_dw_*`, `i_dr_*` chọn. Các buffer khác nhau là các RAM khác nhau nên không tranh nhau; trong cùng một buffer, Main FSM cấp hai bank khác nhau cho hai phía (scoreboard) khi phía tính thực sự dùng buffer đó, được kiểm bằng AS-02. Riêng BIAS: job có `POST_EN` = 0 vẫn mang một giá trị `i_comp_bank_bias` nhưng không đọc BIAS, nên DMA được phép ghi BIAS vào bank đó
- Đọc A/B/BIAS trả dữ liệu ở chu kỳ kế tiếp; bên nhận chỉ dùng các lane hợp lệ. Ghi C với fmt = 0 lấy phần tử c từ `data[32c +: 32]` và `mask[c]` cho phép ghi cả word. Với fmt = 1, lấy byte c từ `data[8c +: 8]`, `mask[c]` cho phép ghi byte đó; phần cao `data[255:64]` bằng 0. Hàng/cột đầu phải hợp lệ và cột đầu là bội của 8. Mask chặn các phần tử ngoài miền tác vụ
- Mỗi yêu cầu đọc DMA được nhận phải tạo đúng một `o_dr_rvalid` ở chu kỳ tiếp theo, theo đúng thứ tự; không có ready cho dữ liệu trả về. Nội dung RAM và các lane không hợp lệ không cần khởi tạo, và được phép chứa X trong mô phỏng

2 Đặc tả thanh ghi

Module không có thanh ghi ánh xạ MMIO. Toàn bộ thanh ghi MMIO nằm trong `gemm_register_file`, xem đặc tả hệ thống baseline. Cấu hình được nhận qua các tín hiệu đầu vào (bank phía tính từ Main FSM, lệnh từ DMA, array và VPU); trạng thái nội bộ, điều kiện chốt và reset được mô tả bên dưới.

3 Mô tả chức năng

3.1 Bố cục logic

Bảng 3.1: Địa chỉ word logic trong mỗi bank

Buffer	Địa chỉ a	Nội dung word
A	{i[4:0], kw[2:0]}, kw = k » 2	Byte e = $A[i][4kw+e]$; lane = k[1:0]
B	{k[4:0], jw[2:0]}, jw = j » 2	Byte e = $B[k][4jw+e]$
BIAS	{j[4:0]}	$bias[j]$
C, INT32	{i[4:0], j[4:0]}	$C[i][j]$ (word 0 … 1023)
C, INT8	{i[4:0], jw[2:0]}	Byte e = $Y[i][4jw+e]$, lane = j[1:0] (dùng word 0 … 255)

- Mỗi phần tử A, B là INT8 có dấu; bốn phần tử liên tiếp theo K (A) hoặc theo N (B) nằm trong một word 32 bit, phần tử có chỉ số thấp ở byte thấp (little-endian, trùng thứ tự byte của DRAM)
- Cùng một buffer C chứa INT32 (khi `POST_EN` = 0) hoặc INT8 (khi `POST_EN` = 1); định dạng do `i_c_wr_fmt` chọn ở mỗi lệnh ghi và do `i_wr_fmt` của DMA chọn khi đọc. SRAM không lưu định dạng
- Quy tắc đóng gói INT8 phía VPU: phần tử c nằm ở `i_c_wr_data[8c+7:8c]`, `i_c_wr_data[255:64]` = 0; `i_c_wr_mask[c]` là byte enable của đúng byte đó

3.2 Tổ chức vật lý và ánh xạ

Bảng 3.2: Sub-bank của mỗi bank (P = 8, Q = 2, LP = 3)

Buffer	Số sub-bank	Sâu × rộng	Sub-bank của a	Địa chỉ cục bộ
A	8	32 × 32 bit	i mod 8 = i[2:0]	{i[4:3], kw} (5 bit)
B	Q = 2	128 × 32 bit	jw mod 2 = jw[0]	{k, jw » 1} (7 bit)
BIAS	8	4 × 32 bit, register file	j mod 8 = j[2:0]	j » 3 = j[4:3] (2 bit)
C	8	128 × 32 bit, ghi theo byte	a mod 8 = a[2:0]	a » 3 = a[9:3] (7 bit)

Bảng 3.3: Kiểm kê bộ nhớ vật lý (NB = 2 bank)

Bộ nhớ	Kích thước một RAM	Số word	Số RAM	Ghi chú
`spm_a_subbank`	1024 bit	32	16	8 sub-bank × 2 bank; một cổng đọc/ghi
`spm_b_subbank`	4096 bit	128	4	2 sub-bank × 2 bank
`spm_bias_subbank_reg`	128 bit	4	16	8 sub-bank × 2 bank; register file
`spm_c_subbank_be`	4096 bit	128	16	8 sub-bank × 2 bank; byte enable

Tổng dung lượng: 16 × 1024 + 4 × 4096 + 16 × 128 + 16 × 4096 = 100 352 bit = 3136 word × 32 bit = 12,25 KiB.

> **QUYẾT ĐỊNH BASELINE**
> **D-28 (kích thước và độ trễ).** Mỗi bank: A 256 word, B 256 word, C 1024 word, BIAS 32 word, tổng 1568 word mỗi bank, 3136 word cho cả hai bank (12,25 KiB). Mỗi sub-bank là SRAM một cổng. Độ trễ đọc đúng 1 chu kỳ; ghi có

hiệu lực ở cạnh clock cuối chu kỳ lệnh; không có backpressure, `o_dw_ready` = `o_dr_ready` = 1.

3.2.1 Truy cập phía tính

Cổng phía tính (b = `i_comp_bank_*`)

- Đọc A (i_0, k): sub-bank r (0 ≤ r < 8) của A[b] đọc địa chỉ {i_0[4:3], k[4:2]}. Lane r = byte k[1:0] của sub-bank r; k[1:0] được chốt để chọn byte ở chu kỳ sau
- Đọc B (k, j_0): sub-bank s (s = 0, 1) của B[b] đọc {k, j_0[4:3]}. Lane c = byte (c mod 4) của sub-bank c/4
- Đọc BIAS (j_0): sub-bank c (0 ≤ c < 8) của BIAS[b] đọc j_0[4:3]; lane c = sub-bank c
- Ghi C INT32 (i, j_0): lane c ghi sub-bank c của C[b] tại {i[4:0], j_0[4:3]}, dữ liệu `i_c_wr_data[32c +: 32]`, cả 4 byte, nếu `mask[c]`
- Ghi C INT8 (i, j_0): word w (w = 0, 1) của đoạn hàng có địa chỉ a = {i, j_0/4 + w}, nằm ở sub-bank s = {j_0[4:3], w}, địa chỉ cục bộ a » 3 = i (chiếm 32 word đầu của sub-bank). Sub-bank s nhận word w = s[0] nếu s[2:1] = j_0[4:3]: dữ liệu `i_c_wr_data[32w +: 32]`, byte enable e = `mask[4w+e]`. Các sub-bank còn lại không được chạm

3.2.2 Truy cập phía DMA

Bảng 3.4: Ánh xạ cổng DMA (a = địa chỉ word logic)

Cổng	Buffer	Sub-bank	Địa chỉ cục bộ
Ghi `i_dw_*`	A, `i_dw_buf` = 0	a[5:3] (= i[2:0])	{a[7:6], a[2:0]}
Ghi `i_dw_*`	B, `i_dw_buf` = 1	a[0] (= jw[0])	a[7:1]
Ghi `i_dw_*`	BIAS, `i_dw_buf` = 2	a[2:0]	a[4:3]
Đọc `i_dr_*`	C	a[2:0]	a[9:3]

- Ghi `i_dw_*`: buffer `i_dw_buf`, bank `i_dw_bank`, cả 4 byte của `i_dw_data`; `i_dw_buf` = 3 là không hợp lệ (AS-01)
- Đọc `i_dr_*`: C[`i_dr_bank`], sub-bank và địa chỉ cục bộ theo bảng. Số sub-bank và bank được chốt để chọn dữ liệu ra ở chu kỳ sau

VÌ SAO KHÔNG BAO GIỜ XUNG ĐỘT
i_0 và j_0 là bội của 8, nên 8 hàng của một đọc A rơi vào 8 sub-bank khác nhau, 8 cột của một đọc BIAS hoặc ghi C INT32 cũng vậy; hai word của một đọc B hoặc ghi C INT8 nằm liền nhau nên rơi vào 2 sub-bank khác nhau. Ở INT8, {i, jw} = 8i + jw và 8i chia hết cho 8, nên sub-bank chỉ phụ thuộc jw. Mô hình kiểm đủ mọi (i_0, k), (k, j_0) và mọi đoạn hàng C.

3.2.3 Chọn bên truy cập

Mux cổng của mỗi sub-bank (X, b, s)

- `dma_sel` = lệnh DMA (ghi `i_dw_*` cho A, B, BIAS; đọc `i_dr_*` cho C) có buffer X, bank b và sub-bank s
- `cmp_sel` = lệnh phía tính tương ứng (`i_a_rd_en`, `i_b_rd_en`, `i_bias_rd_en`, `i_c_wr_en` với byte enable khác 0 cho s) và `i_comp_bank_X` = b
- RAM: `en` = `dma_sel | cmp_sel`; địa chỉ, dữ liệu, `we`, byte enable lấy từ bên được chọn. Nếu cả hai cùng chọn, DMA thắng
- Không cần tín hiệu pha: quyền truy cập do bank và lịch của FSM bảo đảm

QUYẾT ĐỊNH BASELINE
D-12 (trọng tài cổng SRAM). `en` của mỗi sub-bank RAM = `dma_sel | cmp_sel`; khi trùng, DMA thắng. Đồng thời assertion AS-02 (chỉ mô phỏng) khẳng định tình huống này không bao giờ xảy ra khi `i_comp_active` = 1, vì FSM đã cấp bank khác nhau cho hai phía. Lý do: hành vi phải xác định được trong RTL; assertion là cơ chế phát hiện lỗi lập lịch.

3.3 Module con

Bảng 3.5: Tín hiệu giao tiếp của `gemm_spm_ram` (tham số DEPTH, BE)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`i_en`	1	Input	Cho phép truy cập RAM tại cạnh lên. Khi bằng 0, không có lệnh đọc hoặc ghi mới.
`i_we`	1	Input	Chọn thao tác RAM khi i_en = 1. Giá trị 1: ghi; 0: đọc đồng bộ.
`i_be`	4	Input	Cho phép ghi từng byte của word. Khi tham số BE = 1, bit b điều khiển byte b; khi BE = 0, bỏ qua mặt nạ và ghi cả word.
`i_addr`	$\lceil \log_2 \text{DEPTH} \rceil$	Input	Địa chỉ word trong RAM con, từ 0 đến DEPTH − 1. Được lấy mẫu cùng i_en tại cạnh lên.
`i_wdata`	32	Input	Word 32 bit ghi vào RAM khi i_en và i_we cùng bằng 1. Các byte thực sự được cập nhật theo mặt nạ i_be.
`o_rdata`	32	Output	Word 32 bit đọc từ RAM. Hợp lệ một chu kỳ sau lệnh có i_en = 1 và i_we = 0.

Bảng 3.6: Module con theo buffer

Module	Nội dung
`gemm_spm_a`	16 RAM (NB × 8), mỗi RAM 32 × 32 bit; phục vụ đọc vector A và ghi word DMA; chốt k[1:0] và bank để chọn byte trả về.
`gemm_spm_b`	4 RAM (NB × Q), mỗi RAM 128 × 32 bit; phục vụ đọc vector B và ghi word DMA; chốt bank để chọn dữ liệu trả về.
`gemm_spm_bias`	16 RAM (NB × 8), mỗi RAM 4 × 32 bit; hiện thực bằng register file vì mỗi RAM chỉ sâu 4 word, đọc vẫn đồng bộ; đọc vector bias và ghi word DMA.
`gemm_spm_c`	16 RAM (NB × 8), mỗi RAM 128 × 32 bit có byte enable; ghi đoạn hàng INT32/INT8 và đọc word DMA; ghép word theo j_0 khi ghi INT8, chốt bank và sub-bank để chọn dữ liệu đọc.

Bốn module con theo buffer chỉ chứa RAM và thanh ghi dữ liệu nên không có `rst_n`. Việc giải mã buffer (`i_dw_buf`) và thanh ghi `o_dr_rvalid` nằm ở `gemm_local_sram`.

Bảng 3.7: Tín hiệu giao tiếp của `gemm_spm_a`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối.
`i_dw_valid`	1	Input	Ghi một word DMA vào A; bằng `i_dw_valid` ∧ (`i_dw_buf` = 0).
`i_dw_bank`	1	Input	Bank đích của word ghi.
`i_dw_addr`	8	Input	Địa chỉ word logic {i[4:0], kw[2:0]}, 0 … 255.
`i_dw_data`	32	Input	Word ghi, cả 4 byte.
`i_comp_bank`	1	Input	Bank phía tính, bằng `i_comp_bank_a`.
`i_rd_en`	1	Input	Cho phép đọc vector A, bằng `i_a_rd_en`.
`i_rd_row`	5	Input	Hàng đầu i_0, bội của 8.
`i_rd_k`	5	Input	Chỉ số k.
`o_rd_data`	64	Output	Vector 8 byte, lane r tại [8r +: 8]; hợp lệ 1 chu kỳ sau `i_rd_en`.

Bảng 3.8: Tín hiệu giao tiếp của `gemm_spm_b`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối.
`i_dw_valid`	1	Input	Ghi một word DMA vào B; bằng `i_dw_valid` ∧ (`i_dw_buf` = 1).
`i_dw_bank`	1	Input	Bank đích của word ghi.
`i_dw_addr`	8	Input	Địa chỉ word logic {k[4:0], jw[2:0]}, 0 … 255.
`i_dw_data`	32	Input	Word ghi, cả 4 byte.
`i_comp_bank`	1	Input	Bank phía tính, bằng `i_comp_bank_b`.
`i_rd_en`	1	Input	Cho phép đọc vector B, bằng `i_b_rd_en`.
`i_rd_k`	5	Input	Chỉ số k của hàng B.
`i_rd_col`	5	Input	Cột đầu j_0, bội của 8.
`o_rd_data`	64	Output	Vector 8 byte, lane c tại [8c +: 8]; hợp lệ 1 chu kỳ sau `i_rd_en`.

Bảng 3.9: Tín hiệu giao tiếp của `gemm_spm_bias`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối.
`i_dw_valid`	1	Input	Ghi một word DMA vào BIAS; bằng `i_dw_valid` ∧ (`i_dw_buf` = 2).
`i_dw_bank`	1	Input	Bank đích của word ghi.
`i_dw_addr`	5	Input	Chỉ số j của bias, 0 … 31.
`i_dw_data`	32	Input	Word bias INT32.
`i_comp_bank`	1	Input	Bank phía tính, bằng `i_comp_bank_bias`.
`i_rd_en`	1	Input	Cho phép đọc vector bias, bằng `i_bias_rd_en`.
`i_rd_col`	5	Input	Cột đầu j_0, bội của 8.
`o_rd_data`	256	Output	8 bias INT32, lane c tại [32c +: 32]; hợp lệ 1 chu kỳ sau `i_rd_en`.

Bảng 3.10: Tín hiệu giao tiếp của `gemm_spm_c`

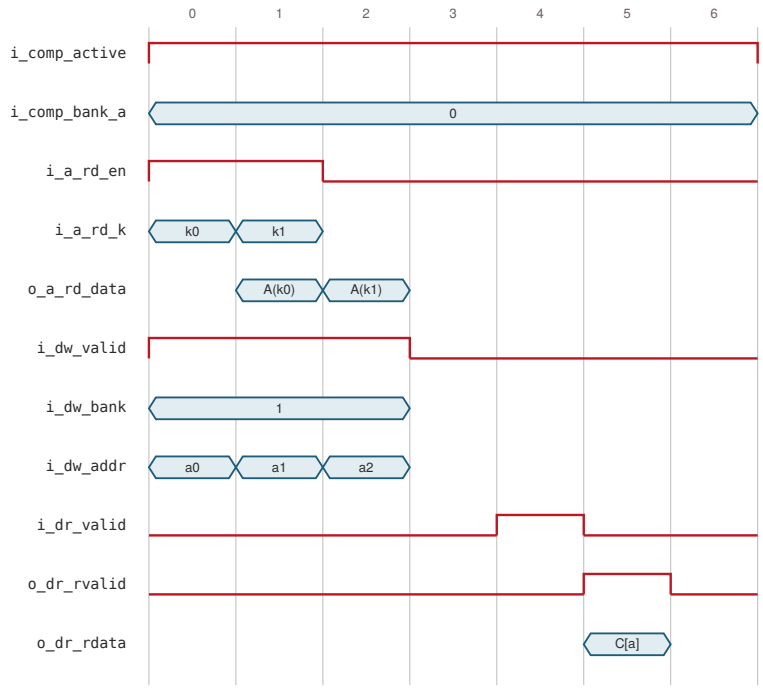
Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối.
`i_dr_valid`	1	Input	Yêu cầu đọc một word DMA từ C.
`i_dr_bank`	1	Input	Bank nguồn của word đọc.
`i_dr_addr`	10	Input	Địa chỉ word logic trong C.
`o_dr_rdata`	32	Output	Word đọc, hợp lệ 1 chu kỳ sau `i_dr_valid`.
`i_comp_bank`	1	Input	Bank phía tính, bằng `i_comp_bank_c`.
`i_wr_en`	1	Input	Ghi một đoạn hàng, bằng `i_c_wr_en`.
`i_wr_fmt`	1	Input	0: INT32; 1: INT8.
`i_wr_row`	5	Input	Chỉ số hàng i.
`i_wr_col`	5	Input	Cột đầu j_0, bội của 8.
`i_wr_data`	256	Input	Dữ liệu đoạn hàng, bố cục như `i_c_wr_data`.
`i_wr_mask`	8	Input	Mặt nạ phần tử, như `i_c_wr_mask`.

```
// gemm_spm_ram: RAM một cổng, đọc đồng bộ (tham khảo)
logic [31:0] mem [DEPTH];
always_ff @(posedge clk)
  if (i_en) begin
    if (i_we) begin
      for (int e = 0; e < 4; e++)
        if (!BE || i_be[e]) mem[i_addr][8*e +: 8] <= i_wdata[8*e +: 8];
    end else
      o_rdata <= mem[i_addr];
  end
```

3.4 Timing

Bảng 3.11: Thời điểm (t = chu kỳ có lệnh)

Cổng	Lệnh	Kết quả
A, B, BIAS đọc	t	dữ liệu ở t + 1
C ghi (VPU), A/B/BIAS ghi (DMA)	t	có trong RAM từ t + 1
C đọc (DMA)	t	`o_dr_rvalid` = 1, dữ liệu ở t + 1



Hình 3.1: Độ trễ đọc cố định 1 chu kỳ và đồng thời hai bank. Phía tính đọc A bank 0 ở chu kỳ 0 và 1 (`i_comp_bank_a` = 0), dữ liệu A(k0), A(k1) ở chu kỳ 1 và 2. Trong cùng các chu kỳ đó DMA ghi ba word vào A bank 1 (`i_dw_bank` = 1), không tranh sub-bank. Ở chu kỳ 4 DMA đọc C bank 1 (phía tính dùng bank C 0); `o_dr_rvalid` và dữ liệu xuất hiện đúng ở chu kỳ 5. Mọi cổng không có backpressure

- Ghi tại chu kỳ t có hiệu lực ở cạnh clock cuối chu kỳ t; đọc cùng địa chỉ ở chu kỳ t + 1 thấy dữ liệu mới
- Đọc và ghi cùng sub-bank trong cùng một chu kỳ bị cấm (cổng RAM một cổng); thiết kế chỉ cho phép khi hai truy cập thuộc hai bank khác nhau

3.5 Reset, hiện thực, kiểm chứng

Reset

- `rst_n` = 0 hoặc `i_soft_clr`: `o_dr_rvalid` về 0. Thanh ghi chọn byte, bank, sub-bank và nội dung RAM không bị reset
- `o_dr_rvalid` là flip-flop trạng thái duy nhất của module: `always_ff @(posedge clk or negedge rst_n)`, nhánh reset và nhánh `i_soft_clr` đều về 0, ngược lại nhận `i_dr_valid`

> **QUYẾT ĐỊNH BASELINE**
> **D-21 (kiểu always_ff).** Flip-flop trạng thái (`o_dr_rvalid`) dùng `always_ff @(posedge clk or negedge rst_n)`; thanh ghi chỉ chứa dữ liệu (thanh ghi chọn byte, bank, sub-bank; RAM; thanh ghi đầu ra RAM) dùng `always_ff @(posedge clk)` và không có giá trị reset. `soft_clr` là xóa đồng bộ, không đi vào chân reset.

Hiện thực

- RAM viết theo mẫu `gemm_spm_ram` để công cụ suy ra block RAM hoặc LUTRAM có đọc đồng bộ. Không được dùng RAM đọc bất đồng bộ cho A, B, C, BIAS
- Không có mux tổ hợp từ `*_rd_en` tới dữ liệu ra; đường ra là: thanh ghi đầu ra RAM → mux bank → mux byte/lane. Thanh ghi chọn bank, byte (k[1:0]) và sub-bank được nạp ở mỗi chu kỳ từ các cổng địa chỉ
- Mỗi buffer trong file riêng; NB và số sub-bank dựng bằng `generate`

Assertion bắt buộc (SVA)

- **AS-01**: `i_dw_valid` → `i_dw_buf` ≠ 3 ∧ địa chỉ trong miền của buffer (A, B < 256; BIAS < 32)
- **AS-02**: khi `i_comp_active` = 1, không sub-bank nào có `dma_sel` ∧ `cmp_sel` cùng chu kỳ; ở mức bank: `i_dw_valid` ∧ `i_dw_buf` ∈ {A, B} → `i_dw_bank` ≠ `i_comp_bank_X` với X = `i_dw_buf`; `i_dw_valid` ∧ `i_dw_buf` = BIAS ∧ `i_bias_rd_en` → `i_dw_bank` ≠ `i_comp_bank_bias`; `i_dr_valid` → `i_dr_bank` ≠ `i_comp_bank_c`. Thành phần BIAS chỉ áp khi phía tính đang đọc bias, vì khi `POST_EN` = 0 Main FSM được phép nạp BIAS cho job sau vào bank trùng `i_comp_bank_bias` (không phải xung đột). Ràng buộc lập lịch đầy đủ cho BIAS nằm ở AS-04 của Main FSM. Đây là assertion của quyết định D-12: RTL vẫn xác định (DMA thắng) nếu vi phạm, assertion phát hiện lỗi lập lịch
- **AS-03**: `i_a_rd_en` ∨ `i_b_rd_en` ∨ `i_bias_rd_en` ∨ `i_c_wr_en` → `i_comp_active`
- **AS-04**: `i_a_rd_row`, `i_b_rd_col`, `i_bias_rd_col`, `i_c_wr_col` là bội của 8 khi có lệnh
- **AS-05**: `o_dw_ready` = `o_dr_ready` = 1 ở mọi chu kỳ; `o_dr_rvalid` bằng `i_dr_valid` của chu kỳ trước
- **AS-06**: `i_c_wr_en` ∧ `i_c_wr_fmt` = 1 → `i_c_wr_data[255:64]` = 0

Kiểm tra chức năng

- Ghi A, B, BIAS qua cổng DMA rồi đọc qua cổng array/VPU: mọi (i_0, k), (k, j_0), j_0 khớp giá trị đã ghi, đúng 1 chu kỳ sau `*_rd_en`
- Ghi C INT32 và INT8 qua cổng VPU với mask ngẫu nhiên rồi đọc qua cổng DMA: byte/word có mask = 0 giữ nguyên giá trị cũ, phần tử có mask = 1 khớp
- Phía tính và DMA truy cập đồng thời hai bank khác nhau của cùng buffer: không sai lệch dữ liệu ở cả hai phía
- Số sub-bank bị chạm trong mỗi truy cập song song bằng đúng bảng ở mục điều kiện phục vụ đồng thời, không sub-bank nào bị chạm hai lần

KIỂM CHỨNG BẰNG MÔ HÌNH

(Mô hình nằm ở `gemm-accel/model/`, đã chạy lại với baseline ngày 12/11/2026; kết quả đạt.) `gemm_sram_model.py`: ánh xạ sub-bank và địa chỉ cục bộ là song ánh với P = 8; 240 cấu hình ngẫu nhiên (M, N, K, bank, NB = 2) nạp A, B, BIAS qua cổng DMA rồi đọc qua cổng array/VPU, ghi C INT32 và INT8 qua cổng VPU rồi đọc qua cổng DMA: mọi giá trị khớp và không truy cập song song nào chạm hai lần một sub-bank.